David Antolick

(570) 926-0234 | david@antolick.ai | antolick.ai | linkedin.com/in/david-antolick | github.com/David-Antolick

Summary

AI/ML engineer building production agentic systems, RAG pipelines, and LLM evaluation infrastructure. Independently built and launched a full-stack RAG platform (climbspeed.com) now live with student pilots and adopted as curriculum at two flight schools, with a custom ReAct agent, hybrid retrieval, and eval-driven iteration achieving 4.86/5.0 correctness across 230 benchmarked questions. Currently deploying multi-agent chatbots and data pipelines in regulated pharma manufacturing at J&J. MS in Computational Biology with hands-on protein language models, distributed GPU training, and molecular ML.

Technical Skills

AI/ML: Agentic AI (ReAct, LangGraph, tool calling, multi-agent orchestration), RAG (hybrid retrieval, BM25, RRF, reranking), text-to-SQL, LLM evaluation (LLM-as-judge, rubric scoring), prompt engineering, knowledge graphs, question generation pipelines
Languages: Python, TypeScript, JavaScript, SQL, Go (Hugo templates), HTML/CSS
Data Engineering: PySpark, Databricks, Delta Lake, Pandas, NumPy, ETL pipelines, JSON/Parquet data modeling
Backend: FastAPI, PostgreSQL, Pydantic, REST APIs, SSE streaming, OAuth/session auth, async Python
ML/DL: PyTorch (DDP, TorchScript), XGBoost, ESM-2, Hugging Face Transformers, scikit-learn, OpenMM
Frontend: Next.js 16, React 19, Tailwind CSS, SWR, SSE parsing, responsive/mobile-first design
Infrastructure: Docker, AWS (S3, CloudFront, Bedrock, AgentCore), Azure Blob Storage, Chroma, vLLM, Git, Linux, CUDA

Experience

ClimbSpeed — Founder & Sole Engineer Aug 2025 – Present
climbspeed.com — Production RAG platform for aviation ground school *Launched April 2026*

- Built, launched, and operate a production RAG-powered aviation ground school platform end-to-end as a solo project: ∼29,000 lines Python, ∼11,900 lines TypeScript/TSX, 1,485 lines Jinja2 prompt templates across 17 templates, 19 PostgreSQL migrations across 21 tables, and a 3-container Docker Compose deployment. Live with real student pilots and adopted as curriculum at two flight schools.
- Designed a custom ReAct agent framework from scratch (no LangChain/LlamaIndex) with an abstract `BaseAgent`, tool-calling loop, state management, and extensible hook system. Iterated from a complex validation agent (78–86% pass) to a minimal 2-tool search+submit design (93–95% pass) — eval data proved measurement-driven simplification outperforms over-engineering.
- Implemented hybrid retrieval: semantic search (Chroma + SentenceTransformers) + BM25 keyword search + Reciprocal Rank Fusion, with agent-controlled multi-round search, parallel queries in a single LLM response, per-question chunk deduplication via content hashing, and round limits tuned by eval data (2-round answers scored 4.82/5 vs 5-round at 4.73/5).
- Built deterministic compute tools (aviation calculator with 9 functions, FAA calendar with 7 functions, 75 unit tests) to fix systematic LLM math failures identified in alpha audit. Tools run in a restricted Python sandbox. System prompt enforces mandatory tool use for numeric and date questions.
- Developed LLM-as-judge eval pipeline with producer-consumer ThreadPoolExecutor architecture, calibrated judging with full untruncated context pass-through (discovered truncating judge context caused 93.9% → 97.3% false-failure swing). Benchmarked at 4.86/5.0 correctness, 99.6% pass rate, 100% citation compliance across 230 questions.
- Shipped a misconception-complete quiz generation pipeline (V5, fifth iteration) driven by a 41.6K-node concept graph with 69K extracted misconceptions: 1 LLM call + 2 agent calls per question at 70–78% yield (up from 40–54% in prior versions), ∼45s wall time. Production bank of ∼4,700 active questions (MCQ + free-response) across Private and Instrument exam types; design grounded in 6 peer-reviewed papers.
- Processed 45+ FAA publications into a searchable knowledge base using Docling hybrid chunking (structure-aware PDF parsing), sliding window tokenization (512 tokens, 128 overlap), and dual indexing into both Chroma (vector) and BM25 (sparse). Hand-curated supplemental FAQs for content that chunks poorly from PDFs.
- Shipped full commercial infrastructure: Stripe Checkout + webhook tier sync + customer portal across four subscription tiers, Google OAuth with PKCE, admin-reviewed CFI verification workflow, GDPR/CCPA-compliant self-service account deletion, per-tier weekly usage quotas, interactive public demo subdomain with 4-layer bot defense (Cloudflare WAF + Turnstile + honeypot + per-IP rate limits), and daily encrypted `pg_dump` backups.

AI and Platform Engineer Sep 2025 – Present

Johnson & Johnson — Carvykti CAR-T Cell Therapy Manufacturing *Remote*

- Designed and deployed a multi-agent chatbot on J&J's internal AI platform (Flowwise/AMP) enabling scientists and engineers to query lentiviral manufacturing data via natural language. Built a Claude 3.5 Sonnet routing agent with domain-aware routing rules, a custom JavaScript SQL executor against a Databricks SQL warehouse, safety guardrails (SELECT-only validation, auto LIMIT 1000), and async query polling.
- Built a PySpark data pipeline ingesting 17 heterogeneous parquet datasets from Azure Blob Storage across two manufacturing sites, normalizing schema conflicts via `unionByName(allowMissingColumns=True)`, constructing a material genealogy graph with iterative DFS (stack-based, memoized), and producing 4 structured JSON knowledge layers + 7 Delta Lake tables powering both the chatbot SQL interface and the Hugo portal.
- Developed a ∼4,600-line CrispML pipeline transforming predictive ML outputs (52 iterations across 6 rounds, 14 CQA outcomes) into hierarchical JSON powering radar, scatter-regression, fishbone, and bi-directional bar visualizations, with S3 discovery (multi-level fallback) and a unified column resolver for cross-round naming inconsistencies.
- Architected a tri-agent risk-intake chatbot (Claude Sonnet 4.5, selected empirically over GPT-4o-mini and Claude Sonnet 3.7 via a custom LLM-as-judge harness) replacing a static intake form with an AI-guided conversational interview. Built a 3-phase forward-only flow (Intake → Assessment → Wrapup) with vector-store RAG for similar-risk calibration over a 57-record register, deterministic 5×5 severity/likelihood heatmap scoring, and CEI (Cause–Event–Impact) statement generation; validated on a held-out scenario set at 4.73/5 average rubric score and exercised by real IMQ quality users.
- Designed the rebuild architecture for a natural-language manufacturing-analytics chatbot (text-to-SQL): diagnosed the legacy multi-agent flow's reliability failures as agent proliferation and specified a single LangGraph `create_react_agent` replacement with a hardened SQL tool layer (`sqlglot` SELECT-only parse gate, automatic LIMIT injection, read-only service principal) and a CI-gated golden-set eval. Profiled a ∼33K-row Databricks quality table to establish the canonical `COUNT(DISTINCT)` counting rule and a curated read-only view.
- Ran an Amazon Bedrock platform evaluation for the team: built a hands-on sandbox of nine runnable examples spanning the Converse API, tool-use agent loops, Bedrock Flows, RAG with Titan embeddings, and managed agents (Strands + AgentCore Runtime/Memory), plus a latency benchmark across invocation paths and a credentials-to-deployment gotcha guide to de-risk a managed-cloud-AI adoption decision.

Projects

REX Voice Assistant | *Python, faster-whisper, openWakeWord, Silero VAD, PySide6, asyncio, CUDA* 2024 – Present

- Built a fully local, wake-word-gated streaming voice assistant (published on PyPI) for hands-free desktop control: an on-device openWakeWord → Silero VAD → Whisper (faster-whisper) → regex-matcher pipeline with a low-latency mode that early-matches partial transcripts under safe/unsafe gating. ∼44 voice commands span YouTube Music, Spotify (OAuth), SteelSeries GG clip capture, app launch/close, and a self-built YouTube-video desktop fork driven over its companion-server API.
- Architected a declarative action registry (`@action` / `ActionSpec`) with slot-based routing — one backend per capability slot, matcher recompiles on backend swap, new integration = one file — backed by CI-gated correctness checks and hard dispatch-latency ceilings (<50 μs/match). Shipped a PySide6 system-tray app with keyring secrets, layered config, per-session p50/p95 + CPU/GPU instrumentation, and Windows CUDA/cuDNN handling with CPU fallback.

Intron Retention Analysis Pipeline | *Python, Pandas, Biopython, Ensembl REST API, Docker* 2024 – 2025

- Built a multi-stage bioinformatics pipeline (Robin Lee Lab, University of Pittsburgh) for downstream IRFinder analysis: processes ∼300K rows/sheet across 4 timepoints, enriches via Ensembl REST/BioMart, reconstructs intron-retained transcripts, and predicts NMD susceptibility (EJC distance rules). Modular OOP with a method-chaining `SequenceProcessor`, pickle caching (50x load speedup), and multi-replicate consensus filtering.

Education

University of Pittsburgh Pittsburgh, PA

M.S. Computational Biomedicine and Biotechnology *Aug 2024 – May 2025*

- Drug-kinase binding prediction: ESM-2 protein language model embeddings (3B params) + XGBoost with Bayesian HPO (scikit-optimize), Spearman 0.624 / ROC AUC 0.759 — embeddings beat hand-crafted k-mer features. Scalable microscopy classification: ResNet-34 over 13 phenotype classes with DDP multi-GPU training (torchrun/NCCL), synchronized BatchNorm, TorchScript export.
- Additional ML: SMILES variational autoencoder for molecular generation (GRU encoder/decoder, 1024-dim latent, TorchScript decoder); protein-ligand MD (OpenMM, CUDA); variant effect prediction (ESM-2 + BLOSUM62, $R^2 = 0.73$); genomic motif prediction (HOMER PSSMs + XGBoost).

Rensselaer Polytechnic Institute Troy, NY

B.S. Computational Biology, Minor in Philosophy *Aug 2020 – May 2024*